

HANDOFF BRIEF

# Ledgr — what we built, what we used, how to continue

Give this PDF (or its text) to GPT or Claude as the first message, then state the next task. This is the full conversation state for the Ledgr / ReconAI Streamlit dashboard as of 22 August 2026.

## 1. What this project is

Ledgr (repo name ReconAI) is a Razorpay Buildathon Track 04 project: an **internal finance-ops reconciliation dashboard**, not a marketing site. It takes orders (what you sold) and settlements (what you were paid), matches them line by line, accounts for gateway and COD fees, ages cash-on-delivery remittances, and leaves only the differences a human should touch.

The product promise: **the engine classifies; the dashboard only displays and lets a person act. No LLM invents rupees.**

User location for date logic: Dhanbad / India. Wall-clock “today” in the app is **22 August 2026** (Asia/Kolkata). Do not let models assume 2024.

## 2. Where the code lives

Real project: `/home/user/reconai`. Ignore leftover scratch at `/home/user/recon`.

|                                     |                                                                 |
|-------------------------------------|-----------------------------------------------------------------|
| <code>app.py</code>                 | Streamlit frontend. This is what we iterate on.                 |
| <code>engine.py</code>              | 5-tier waterfall + Tier 0 COD pre-check. DO NOT MODIFY.         |
| <code>config.py</code>              | Paise helpers, fee bands, COD windows, reason legend, priority. |
| <code>gen_data.py</code>            | Synthetic data generator. DO NOT MODIFY.                        |
| <code>validate_data.py</code>       | Asserts labels vs config. DO NOT MODIFY.                        |
| <code>customers.csv</code>          | Additive contact directory (phone, email, city).                |
| <code>review_log.csv</code>         | Append-only human resolutions.                                  |
| <code>data/orders.csv</code>        | 259 demo orders. Do not edit.                                   |
| <code>data/settlements.csv</code>   | 231 demo settlements. Do not edit.                              |
| <code>data/ground_truth.csv</code>  | 265 labelled expected outcomes.                                 |
| <code>data/run_results.csv</code>   | Engine CLI dump only. App must not read this as source.         |
| <code>.streamlit/config.toml</code> | Light theme, bind 0.0.0.0:8501, fileWatcherType=poll.           |

Design leftovers (not runtime): `demo.html`, `index.html`, `mockup.py`, `dashboard_mockup.svg`, `login_mockup.*`, `FEATURES.md`, `PROJECT_MAP.md` (`PROJECT_MAP` is stale — it still says `app.py` is not built).

## 3. Stack we used

|                    |                                                                     |
|--------------------|---------------------------------------------------------------------|
| Language           | Python 3.13                                                         |
| UI                 | Streamlit 1.62 (single app, no multipage, no auth)                  |
| Data               | pandas 2.2, openpyxl 1.x for Excel upload                           |
| Money              | Integer paise via decimal in config.py — never float math           |
| Dates              | datetime + zoneinfo Asia/Kolkata                                    |
| Voice / LLM APIs   | None. Ask Ledgr is local/deterministic.                             |
| DB / Docker / auth | None. Files + session_state only.                                   |
| Theme              | Light only. Accent #1A56DB / #2D5BFF, ink #0C0E12.                  |
| Preview host       | Streamlit on 0.0.0.0:8501. Health / _stcore/health ≠ code reloaded. |

## Verify UI

streamlit.testing.v1.AppTest — do not import app.py as a module.

## Run command

```
cd /home/user/reconai
pip install streamlit # packages do not persist across sandbox recycle
streamlit run app.py --server.address 0.0.0.0 --server.port 8501 \
  --server.headless true --server.enableCORS false \
  --server.enableXsrfProtection false --server.fileWatcherType poll
```

## 4. Architecture that must stay true

```
CSV data
→ engine.load()
→ engine.reconcile() # real classification
→ session_state.results
→ app.py Streamlit dashboard

Frontend joins (never change engine for these):
order_date / customer / mode ← orders.csv on record_id
settled_on                    ← settlements.csv
phone / email / city          ← order_id → orders.customer_name → customers.csv
resolved flag                 ← review_log.csv (append-only)
```

Run reconciliation must call real load()+reconcile(), optionally after soften\_inputs() so uploaded files missing optional columns do not KeyError. Never treat run\_results.csv as the app data source.

**DO NOT TOUCH engine.py, gen\_data.py, validate\_data.py, orders.csv, settlements.csv, or run\_results.csv. Do not add OpenAI/Gemini/Claude APIs, .env, or st.secrets.**

## 5. Engine waterfall (frozen — for understanding only)

config.RUN\_DATE is 2026-09-01 for reproducible CLI scoring. The app overrides engine.RUN\_DATE with the dashboard “Reconcile as at” date (capped at today in India).

- **Tier 0** — COD, no remittance yet. Age vs as-at: 0–7 wait, 8–14 approaching, 15+ overdue exception.
- **Tier 1** — gateway\_ref\_id + amount exact → AUTO\_CLEARED.
- **Tier 2** — ref match, shortfall inside fee\_band (online 2%/■3 or COD 2.5%/■50).
- **Tier 3** — ref match, amount off-band → ai\_diagnose(facts). Model never computes money.
- **Tier 4** — no ref → bank\_utr fallback, then classify.
- **Tier 5** — unmatched / ambiguous / unlinked bank credit (STL-X\*: received > 0, expected = 0).

CLI: python3 engine.py → 265 records scored, PERFECT vs ground\_truth. Demo: 259 orders (2026-08-01→2026-09-01), 231 settlements (2026-08-02→2026-08-31). Modes: COD, CARD, WALLET, BANK\_TRANSFER, UPI, NETBANKING.

## 6. What we built in this conversation (in order)

All of this is in app.py plus two additive files. Engine and source CSVs were not changed.

### 6.1 Shell dashboard

Streamlit app importing load/reconcile. Demo vs upload. Light theme. fileWatcherType had been none (edits invisible) — set to poll. Dark theme was tried and rejected (contrast). Marketing-site rewrite was rejected — stay internal.

### 6.2 Pre-run inspection

After source choice, show real order/settlement counts and totals, preview tables, then Reconcile. No fake 0 Matched/Waiting before the first run.

### 6.3 Live as-at date

COD ageing is live against the picker. Waiting records are COD with no UTR yet, not hardcoded. Picker max = today India (2026-08-22). Future dates blocked. Changing as-at re-runs if already reconciled. Button label is always “Run reconciliation”,

never “Run again”.

## 6.4 Settlement graph + money + plain-language buckets

Bars from settlements.settled\_on. Window = 1st of as-at month → as-at (no future money). Hover tooltips. Slider filters the table. Money: Expected / Received / Gap. Outcomes: Matched / Waiting / Needs a look / Show everything. Reason cards: Unmatched, Remittance overdue, Partial payment, Large variance flagged by AI. Click filters + scroll to the record list.

## 6.5 Home scroll wordmark

Entry page: large “Ledgr” (no logo icon) shrinks to top-left on scroll via CSS animation-timeline. Flat light background, no grid/gradient.

## 6.6 Ask Ledgr (local)

No Gemini/GPT. Deterministic reply() on the current run. Order ID and later S.No. lookup. Aggregates for risk / review / COD / summary. Never invent figures.

## 6.7 Manual resolution

Needs a look → Mark as resolved → required note → append review\_log.csv. Row keeps original tier/reason + green “Resolved manually”. Live Needs a look count and at-risk exclude resolved. Show resolved = only resolved rows. Hide resolved (default) = only open. Toggle sits under Find a record.

## 6.8 Contacts

customers.csv for 15 demo people. Lookup is only order\_id → orders.customer\_name → customers.csv. No name search. Inline  Contact expand on Needs a look rows. Unlinked STL-X: “No order on file — cannot identify sender”.

## 6.9 Upload hardening

User CSVs crashed on missing customer\_name, source, narration. soften\_inputs() fills those empty. run\_engine() calls reconcile(\*soften\_inputs(\*load())). Required columns still validated.

## 6.10 Navigation + pacing

← next to Ledgr calls change\_source() (same as upload Back). Demo click: 1.5s “Loading demo data” then inspection at top. Reconcile loading animation was added then removed — click goes straight to results. execute() is cached on (datadir, as-at); do not nonce-bust every click.

# 7. Current user-facing flow

```
Home (scroll) → Use demo data (1.5s wait) or Upload
→ Pre-run: 259/231 (or user files), date picker ≤ today
→ Run reconciliation (instant after cache; land at top)
→ Results:
    settlement bars (1st → as-at)
    money + “N resolved · still at risk”
    outcome + reason cards (click → filter + scroll)
    find record
    Show resolved / Hide resolved
    table with S.No., Resolve, Contact
    Ask Ledgr (bottom right)
→ ← Back resets to home (change_source)
```

# 8. Important session keys

source, datadir, results, runs, ms, asat, dfrom, dto, pick, reason, q, mode, chat\_open, ask, focus, resolve\_id, contact\_id, hide\_resolved (default True = open only), land\_top, pending (demo wait), jump, jumps.

# 9. Lessons (do not repeat these bugs)

- fileWatcherType=None meant edits never appeared. Use poll. Health=ok is not proof of reload.
- Do not import app.py outside Streamlit (session\_state is None → crash). Use AppTest.

- Outcome cards: HTML face + invisible overlay ate clicks. Keep the Streamlit button in document flow; face sits on top with pointer-events:none.
- components.html scroll scripts must change (nonce) or Streamlit will not re-run them.
- Table header must share the same column split as rows, or money headings sit over the action buttons.
- Contact + Resolve stacked doubled row height. Keep them on one row; Contact is a person icon + tooltip.
- Ask Ledgr must not search by customer name. Follow-up keywords like “it” stole “credits” — use word boundaries; check unlinked/resolved aggregates first.
- Fake 100/200/259 progress over seconds while the engine is 1ms is dishonest. Loading UI was removed.
- A full marketing-site rewrite was rejected. Stay an internal dashboard. Light mode only.

## 10. How Ask Ledgr resolves a person

S.No. 3 / sno 3 / #3 / serial 3  
 → row N in the currently visible filtered table  
 → that row's record\_id (order\_id)  
 → orders.csv customer\_name  
 → customers.csv phone, email, city

ORD-00023 / “who do I contact for ORD-00023”  
 → same chain. No name search.

STL-X0002 (expected 0, received > 0)  
 → unlinked bank credit, not a data error, no customer.

## 11. Demo script for a judge

- Use demo data → 259 orders, 231 settlements (real CSV totals).
- Run reconciliation → live buckets (as-at 22 Aug, not config 1 Sep).
- Move as-at earlier/later → waiting vs overdue COD changes. Not hardcoded.
- Click Waiting / Needs a look → list filters and scrolls.
- Contact icon on a row → name, phone, email, city via order\_id.
- Mark resolved + note → Needs a look count and at-risk drop; row stays with tag.
- Ask Ledgr: S.No. 1, ORD-00023, how much is at risk, STL-X0002.

## 12. What to tell GPT or Claude to do next

Start from this brief. Work only in /home/user/reconai. Prefer app.py. Do not touch the engine or source CSVs. Keep it a single internal dashboard. After UI edits, restart Streamlit (poll watcher; do not trust HTTP 200). Verify with AppTest when you can. Be concise with the user; show evidence, not essays.

**Judge one-liner: Two CSVs go into a deterministic waterfall. The dashboard shows only what reconcile() returned. A human contacts the person on that order\_id and marks it handled. The engine's classification stays; live at-risk updates. No API key, no hallucinated rupees.**

---

*End of handoff. Paste the text of this PDF, or attach the file, as the first message to GPT or Claude, then give the next instruction.*